



Editorial

Tasks, test data and solutions were prepared by: Vito Anić, Josip Klepec, Dorijan Lendvaj, Martina Licul, Patrick Pavić, and Petar Sruk.

Implementation examples are given in attached source code files.

Task Estimathon

Prepared by: Vito Anić

Necessary skills: arrays, for loop, if statement

First, if $n < m$, the answer is **NE**. Why? Because for each table we need to use chairs of the same color. Therefore, we can use n different chair colors at most. Domagoj wants us to use every chair color. If $n < m$ that is impossible.

If we have less than 4 chairs of the same color the answer is also **NE**. Why? Let us look at these two cases: We do not use that color (Domagoj is sad), we pair these chairs with chairs of some other color (Paula is sad). And we want to make them both happy :).

Third and final check: can we fill all the tables with chairs? For some color i , we can fill $a_i/4$ (rounded down to the nearest integer) tables. If we sum up the values for each color, we will get the maximum number of tables we can fill with chairs. Now if that number is smaller than n , the answer is **NE**, otherwise the answer is **DA**.

Task Dirigent

Prepared by: Josip Klepec

Necessary skills: arrays

For first subtask, we can try breaking the circle at each position and then check does it result in a sorted sequence. Complexity of such approach is $O(qn^2)$.

For second subtask we can see that it is sufficient to try and break the circle at position which will result in 1 being first in the sequence. In this approach we check if sequence is sorted only once and not n times. Complexity is therefore $O(qn)$.

For all points let's focus on neighbours (all pairs of students that are holding hands). There is also n such pairs. Let's call a pair **good** if the one on the right (in clockwise direction) has bigger number, and **bad** otherwise.

Observe that the condition from the task holds iff there is exactly 1 **bad** pair. For full points we will try to maintain number of **bad** pairs.

After each swap, at most 4 pairs of neighbours change so we can easily maintain that number. Complexity of such approach is $O(n + q)$ because we have to initially count number of **bad** pairs and also update it after each swap.

Task Baltazar

Prepared by: Dorijan Lendvaj

Necessary skills: Dijkstra's algorithm, dynamic programming

For the first subtask, we can use Dijkstra to calculate the distance between cities 1 and n , and after that for each road we can calculate the distance between cities 1 and n if the length of that road is increased by 2 km and see if that distance is higher than the initial distance by 1 km, for a total time complexity of $O(m^2 \log m)$.



For the second subtask, the guarantee of that subtask tells us that, no matter which road's length is increased by 2 km, due to the road between 1 and n the length of the shortest path will be increased by at most 1 km, so we're looking for roads whose increase of length by 2 km changes the length of the shortest path between 1 and n . So, we're looking for roads which are on all shortest paths between 1 and n . If d_i is the length of the shortest path between 1 and i , then let's make a directed graph in which we put an edge between a_i and b_i if $d_{a_i} + w_i = b_i$, then each path between 1 and n in the new graph corresponds to a shortest path between 1 and n , so we need to find the edges that are in all paths from 1 to n . That can be done by comparing the number of paths from 1 to n with the number of paths from 1 to n that pass through that edge. We can use dynamic programming to calculate the number of paths from 1 to each vertex, as well as the number of paths from each vertex to n , then the number of paths between 1 and n that pass through an edge between x and y is equal to the product of the number of paths from 1 to x and the number of paths from y to n . Those numbers are very large, but we can look at them modulo 5 randomly generated prime numbers between 1 and 10^9 . Let's calculate the probability that this method says that the total number of paths and the number of paths passing through a given edge is equal, even though in reality it isn't. For that to happen, the difference between those 2 numbers must be divisible by all 5 prime numbers we chose. It can easily be seen that the number of paths between 1 and n is $< 2^n$, so their difference is $< 2^n$, so for $n \leq 3 \cdot 10^5$ it has less than $2 \cdot 10^4$ different prime factors. There are more than $5 \cdot 10^7$ prime numbers up to 10^9 . So, the chance that it's divisible by all 5 primes is at most $\frac{\binom{2 \cdot 10^4}{5}}{\binom{5 \cdot 10^7}{5}} \approx 10^{-17}$, and the probability that one of the m edges gives a wrong answer is that multiplied by m , so about $3 \cdot 10^{-12}$. In practice, the probability is much smaller because it's impossible to make test cases such that the difference in the number of paths are whichever numbers we want; the number of ways to choose m numbers $\leq 2^n$ is about $(2^n)^m$, and the number of possible graphs is about $(n^2)^m$, which is much smaller; in practice just 1 prime number is enough.

For the third subtask, we also need to check for each road whether it's on all paths from 1 to n of length $d_n + 1$. To do that, we can check if it's on all shortest paths from 1 to n with a length of a different parity than d_n and making sure that that shortest path has a length of $d_n + 1$. To do that, we can replace each vertex and edge with 2 vertices and 2 edges; the new vertices will be one for paths of odd length and one for paths of even length, and we can then do the almost the same thing as for the second subtask. Then a road increases the length of the shortest path by 1 if and only if the length of the shortest path between 1 and n with a length of different parity than d_n is $d_n + 1$, if it's on all shortest paths between 1 and n and it's not on all shortest paths between 1 and n with a length different parity than d_n .

Task Bomboni

Prepared by: Patrick Pavić

Necessary skills: dynamic programming, number theory

This problem can be solved with dynamic programming. The easiest approach is to keep the product in the current state, but the product can become really big. So the smarter approach is instead of the product p , keep $\gcd(p, k)$. Notice that a number p is divisible by k if and only if $\gcd(p, k) = k$. Notice the following (the proof is left as an exercise to the reader):

$$\gcd(pq, k) = \gcd(\gcd(p, k) \cdot \gcd(q, k), k)$$

So let us define $dp_{i,j,g}$ as the number of paths starting at the upper left cell and ending at the cell (i, j) such that the greatest common divisor of the product of the numbers on the candy and k is equal to g . Now it is easy to see the relation. We shall analyse the time complexity. Notice that all numbers smaller than 10^6 have at most 300 divisors, which means at most 300 different values of $g = \gcd(p, k)$. So the number of states is $O(300 \cdot N^2)$ which is enough with a constant relation.



Task Skrivača

Prepared by: Petar Sruk

Necessary skills:

For first subtask notice that if $a_v = v$ as soon as Marin enters room v the game ends. Also if a_v is adjacent to v Marin can first go to room v and in the next move enter room a_v and the game ends. In the only remaining case it is enough to check only the last step of the game, if in his last move Marin goes from room u to room v we need to check if Luka can get from room a_u to room a_v without passing through room v . For that it is enough to remove vertex v from graph and check if a_u and a_v are connected with a standard dfs algorithm. Now that we know all possible last moves we can easily calculate how many moves are necessary to end the game from each starting room using bfs. Since there are $2m$ possible last moves (one for each direction of each edge), and dfs and bfs run in $O(n + m)$ time complexity, the full time complexity of this approach is $O(m(n + m))$.

In the second subtask the graph is a tree so to check if a_u is connected to a_v after removing vertex v we can use LCA which has time complexity of $O(\log(n))$.

Consider $a_v \neq v$ nor a_v is adjacent to v , the game can only end in articulation points. Furthermore, since for each articulation point it is fairly easy to construct hiding strategy for which the game can end in that vertex, we conclude that the second part of the third subtask is equivalent to: "There are at most 5 articulation points." We can find articulation points using dfs spanning tree in time complexity of $O(n + m)$. Since there are at most 5 of them, for each articulation point v we can check if a_v is connected to a_u with dfs similar to the first subtask.

For complete solution we need to check if vertices a_v and a_u are connected after removing vertex v faster. In dfs spanning tree consider times of entry for each vertex in the same connected component with respect to some articulation point v . It holds that times of entry of every vertex in some connected component that is not connected to the root of dfs spanning tree form a continuous interval. The claim follows from the fact that dfs visits all nodes from some connected component before recursively backtracking back to the articulation point. Now we can determine in which interval some node belongs using binary search, and if it doesn't belong to any interval we know it is in the component connected to the root of dfs spanning tree. Time complexity of such approach is $O(n + m \log(n))$. For implementation details refer to attached source code.

In the attached source codes there will also be an alternative solution to this problem.